

LOW LEVEL DESIGN (LLD)
ON
IaC Provisioning for Agriculture System

Course Name : Devops

Institution Name : Mediacaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s) :

S.NO	Student Name	Enrolment No
1.	Pawani Shah	EN22CS301700
2.	Praniti Kshirsagar	EN22CS301726
3.	Pranjal Patidar	EN22CS301730
4.	Prafulla Kumar Verma	EN22CS301715
5.	Ritik Thakur	EN22CS301811

Table of Contents

1.Introduction

1.1 Scope of the document

1.2 Intended Audience

1.3 System overview

2.System Design

2.1 Application Design

2.2 Process Flow

2.3 Information Flow

2.4 Component Design

2.5 Key Design Considerations

2.6 API Catalogue

3. References

1. Introduction

1.1 Scope of the Document

This document provides a **detailed low-level design of the Krishi AI Assistant system**, focusing on:

- Docker-based containerization
- Image creation and container lifecycle
- Deployment architecture and execution flow
- Runtime networking and service communication
- Integration with external AI services (Groq APIs)

The scope includes **actual implementation details**, such as:

- Docker images created
- Containers running in the system
- Port mappings and deployment validation

1.2 Intended Audience

- DevOps Engineers
- Backend Developers
- Deployment Engineers
- System Architects
- QA Engineers

1.3 System Overview

Krishi AI Assistant is a **containerized AI-powered agricultural system** designed to:

- Analyze crop images and farmer inputs
- Provide disease diagnosis and treatment suggestions
- Estimate market price and insurance options

Deployment Overview

The system is deployed using:

- **Frontend Container (React Application)**
- **Backend Container (FastAPI Application)**
- **External AI APIs (Groq – LLM + Vision)**

Key Deployment Characteristics

- Fully containerized architecture
- Independent services (frontend & backend)

2. System Design

2.1 Application Design

Architecture Type

- Client-Server Architecture
- Containerized deployment model

Deployment Architecture

```
User Browser
  ↓
Frontend Container (React - Port 3000)
  ↓
Backend Container (FastAPI - Port 8000)
  ↓
External Groq APIs
```

Containerization Strategy

- Each service is packaged as a Docker image
- Containers are instantiated from these images
- Services are isolated but communicate via network

2.2 Process Flow :

A. Build-Time Flow

Source Code

↓

Dockerfile Execution

↓

Docker Image Creation

↓

Images Stored Locally

B. Run-Time Flow

Docker Compose Up

↓

Containers Created

↓

Docker Network Initialized

↓

Ports Exposed

↓

Application Ready

C. Request Processing Flow

1. User interacts with frontend
2. Frontend sends API request to backend
3. Backend processes request
4. Backend calls Groq AI APIs
5. Response returned to frontend

- 6. Output displayed to user

2.3 Information Flow

Input

- Crop image
- Crop details (location, type, issue)
- Voice/text input

Processing

- Backend API processing
- AI model interaction (Groq APIs)

Output

- Disease identification
- Suggested treatment
- Urgency level
- Market price estimation
- Insurance information

2.4 Components Design

1. Frontend Component

Docker Image:

- agriculture-support-triage-agent-frontend

Details:

- Built using React.js
- Lightweight image (~63MB)

Container Configuration:

- Running container
- Port mapping : 0.0.0.0:3000 → 80/tcp

Responsibilities:

- User interface
- Input collection
- API communication

2. Backend Component**Docker Image:**

- agriculture-support-triage-agent-backend

Details:

- Built using Python + FastAPI
- Handles AI integration

Image Size:

- ~1.03 GB

Container Configuration:

- Running container
- Port mapping: 0.0.0.0:8000 → 8000/tcp

Responsibilities:

- API handling
- Business logic
- AI processing
- Authentication

3. Supporting Component (Portainer)**Docker Image:**

- portainer/portainer-ce

Purpose:

- Container monitoring

- UI-based Docker management

4. Docker Images

< Agriculture docker image.png

```
ubuntu@ip-172-31-39-36:~/Agriculture-Support-Triage-Agent$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
agriculture-support-triage-agent-frontend	latest	cfc253319e48	18 minutes ago	63.4MB
<none>	<none>	d1d8bcf9a60a	27 minutes ago	63.4MB
<none>	<none>	1d4ce9c64984	46 minutes ago	63.4MB
agriculture-support-triage-agent-backend	latest	8e1d126b0661	48 minutes ago	1.03GB
portainer/portainer-ce	latest	2ad8d683056c	2 weeks ago	183MB

```
ubuntu@ip-172-31-39-36:~/Agriculture-Support-Triage-Agent$
```

5. Docker Containers (Actual Running State)

< Agriculture docker container.png

1/1

```
ubuntu@ip-172-31-39-36:~/Agriculture-Support-Triage-Agent$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
ee031cf4e08	portainer/portainer-ce:latest	"/portainer"	5 minutes ago	Up 5 minutes	9000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp, 0.0.0.0:8001->8000/tcp, [::]:8001->8000
262f3d343fc	agriculture-support-triage-agent-frontend	"/docker-entrypoint..."	17 minutes ago	Up 17 minutes	0.0.0.0:3000->80/tcp, [::]:3000->80/tcp
cb244cd0169d	agriculture-support-triage-agent-backend	"unicorn main:app --..."	45 minutes ago	Up 45 minutes	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp

```
ubuntu@ip-172-31-39-36:~/Agriculture-Support-Triage-Agent$
```

2.5 Key Design Considerations

1. Container Isolation

- Frontend and backend run independently
- No dependency conflicts

2. Port Mapping

- Frontend → 3000
- Backend → 8000
- Ensures external accessibility

3. Environment Configuration

- Managed via .env file

- Secure API keys (Groq API)

4. Scalability

- Containers can be scaled independently
- Backend scaling for high load

5. Performance Optimization

- Lightweight frontend image
- Backend optimized using Python base image

6. Fault Tolerance

- Containers can restart automatically
- Logs available for debugging

2.6 API Catalogue

Authentication APIs

- POST /signup
- POST /login

Core APIs

- POST /analyze
- GET /market-price
- GET /insurance

System APIs

- /docs (Swagger UI for testing APIs)

3. References

- Docker Official Documentation
- FastAPI Documentation
- React Documentation
- Groq API Documentation